Fuel Management System (FMS) – Project Requirement Sheet / Proposal

Course: Advanced Web Technology (API Based Project)
Project Type: Full Stack (Backend API + Frontend)
Backend: NestJS + TypeScript
Frontend: React + TypeScript
Database: PostgreSQL + TypeORM
Authentication: JWT

1. Project Overview

Develop a secure, RESTful backend API using NestJS + TypeScript for a Fuel Management System with 3 roles: User, Distributor, and Admin. The system will support vehicle registration approval, fuel request workflow, distributor stock management, depot stock management, and a global fuel limit policy controlled by admin.

Main Focus Areas

- JWT authentication with role-based authorization (RBAC)
- User registration/login (email or phone)
- Distributor registration/login (admin approval required)
- Vehicle registration request (admin approval required)
- Fuel request flow: User → Distributor Accept/Deny → Complete fueling
- Stock flow: Depot → Distributor allocation (admin controlled)
- History/audit logs for users, distributors, and admin
- **Fuel quota policy**:
    - Admin sets limit globally by vehicle type (Bike/Car)
    - Quota is enforced against vehicle number
    - Admin can change limit liters + limit days
    - When admin changes a limit, a new cycle starts immediately from that change time

2. User Roles & Responsibilities

2.1 User (Vehicle Owner/Driver)

- Register with: full name, NID number, driving license number, phone, date of birth, email, password

- Login using email or phone + password
- Add vehicles (vehicle number is unique) with document upload (PDF/Image)
- View vehicle list & approval status
- Request fuel for an approved vehicle by selecting:
    - vehicle number
    - distributor (select from list or QR selection)
    - fuel type
    - requested liters
- View profile and fueling history

2.2 Distributor (Fuel Pump)

- Register pump with: pump name, unique pump ID, address, owner name, documents, email, password
- Admin approves registration before distributor can access system
- Dashboard:
    - View stock by fuel type (Octane/Diesel/Petrol)
    - View incoming fuel requests
    - Accept or deny requests
    - Complete fueling and generate transaction records
- View fuel delivery history

2.3 Admin

- Approve/reject vehicle registrations
- Approve/reject distributor registrations
- Manage depot stock
- Allocate fuel from depot to distributors
- Set and update global limit policy (days + liters) per vehicle type
- View all histories (users, distributors, vehicles, allocations, transactions)

3. Core Functional Requirements (Must-have Endpoints)

#	Feature	HTTP Method	Endpoint Example	Description / Expected Behavior	Priority
1	User Registration & Login	POST	/auth/register/user, /auth/login	Register user; login with email/phone + password; returns JWT	Must
2	Distributor Registration	POST	/auth/register/distributor	Distributor registers; status PENDING until admin approval	Must
3	Get current profile	GET	/auth/me	Protected route — returns authenticated profile	Must
4	Add Vehicle (request)	POST	/vehicles	User submits vehicleNo (unique), vehicleType, owner info, papers upload; status PENDING	Must
5	My Vehicles	GET	/vehicles/my	User sees vehicle list + approval status	Must
6	Admin approves vehicle	PATCH	/admin/vehicles/:vehicleNo/status	Admin sets APPROVED/REJECTED	Must
7	Admin approves distributor	PATCH	/admin/distributors/:id/status	Admin sets APPROVED/REJECTED	Must
8	Create fuel request	POST	/fuel-requests	Validate: vehicle approved + remaining quota + request created as PENDING	Must
9	Distributor pending requests	GET	/fuel-requests/distributor/pending	Distributor sees pending requests	Must
10	Distributor accept/deny	PATCH	/fuel-requests/:id/decision	Accept/deny request; if accept check stock	Must

	Distributor completes fueling	PATCH	/fuel-requests/:id/complete	Deduct stock + create history transaction	Must
	User history	GET	/history/my	User sees: vehicleNo, date, liters, fuelType, distributor	Must
	Distributor history	GET	/history/distributor	Distributor sees completed deliveries	Must
	Admin creates/updates global limit cycle	POST	/admin/limit-cycles	Admin sets limitDays + maxLiters for BIKE and CAR; new cycle starts immediately	Must
	Get active global limits	GET	/limits/active	Returns current active limits for BIKE/CAR	Should
	Remaining quota for vehicle	GET	/limits/vehicles/:vehicleNo/remaining	Remaining liters for that vehicle in current cycle	Should
	Depot stock management	GET/POST	/admin/depot-stock	Admin view/update depot stock	Must
	Allocate fuel to distributor	POST	/admin/allocations	Deduct depot stock + add distributor stock (atomic)	Must

4. Fuel Limit Policy (Final Business Logic)

4.1 Global by Vehicle Type

Admin sets:

- **BIKE:** maxLiters, limitDays
- **CAR:** maxLiters, limitDays

4.2 Enforced Against Vehicle Number (Shared Vehicle Rule)

Fuel usage is counted by **vehicleNo**, so if two users request fuel for the same vehicleNo, they share the same quota.

4.3 New Cycle Starts When Admin Updates (Option 2)

When admin creates/updates limits:

- A new cycle starts at the update time (startsAt)
- Cycle ends at endsAt = startsAt + limitDays

To validate a request, calculate used liters as:

- Sum of **COMPLETED** transactions for that vehicleNo within [startsAt, endsAt] Remaining quota = maxLiters - usedLiters

If requested liters > remaining quota → reject with proper error.

5. Database Entities & Relationships (Mandatory)

Mandatory Entities (minimum)

- User
- Distributor
- Vehicle
- FuelRequest
- FuelTransaction (History)
- DistributorStock
- DepotStock
- Allocation
- LimitCycle (Bike/Car)

Relationship Types Demonstrated

- One-to-Many / Many-to-One (User → Vehicles, Distributor → Requests)
- One-to-One (FuelRequest → FuelTransaction)
- Many-to-One (Allocation → Distributor, Allocation → Admin)

6. Technical Requirements

Category	Requirement	Mandatory
Language	TypeScript	Yes

Framework	NestJS (latest stable)	Yes
Auth	JWT + @nestjs/passport	Yes
Authorization	Role-based Guards	Yes
Database	PostgreSQL + TypeORM	Yes
Validation	class-validator + ValidationPipe	Yes
Documentation	Swagger (nestjs/swagger)	Yes
Password Hash	bcryptjs	Yes
File Upload	PDF/Image upload for vehicle & distributor docs	Yes
Error Handling	HttpException + global filter	Yes
Modules	auth, users, distributors, vehicles, fuel-requests, stock, limits, admin, history	Yes
Version Control	GitHub repo + README	Yes
Testing	Jest (6–10 tests recommended)	Recommended

7. Deliverables Checklist

- Public GitHub repository
- README with setup instructions + .env.example
- ERD / schema diagram (image or text)
- Swagger UI working
- Seed/demo data
- Screenshots of:
    - admin approvals
    - fuel request flow
    - history pages
- Optional short demo video